Dossier d'exploitation (DEX) — FileFlow

Titre	Dossier d'exploitation (DEX) — Plateforme FileFlow
Version	1.1
Date	27 octobre 2026
Statut	Validé
Propriétaire	Driss OUAAZIZ — Software Architect
Classification	Interne

Table des matières

1. Objet et périmètre
2. Architecture de déploiement
3. Prérequis
4. Procédure d'installation et première configuration
5. Configuration de référence
6. Supervision
7. Sauvegarde et restauration
8. Plan de reprise et de continuité (PRA/PCA)
9. Gestion des montées de version
10. Sécurité d'exploitation
11. Runbooks
12. Références

1. Objet et périmètre

Le présent dossier décrit les modalités d'exploitation de la plateforme **FileFlow**, application de gestion et de partage de fichiers auto-hébergée. Il s'adresse aux équipes chargées du déploiement, de la supervision, de la sauvegarde et du maintien en condition opérationnelle (MCO) de la solution.

Le périmètre couvre l'installation, la configuration, la supervision, la sauvegarde/restauration, la continuité de service, les montées de version et les procédures de traitement d'incident (runbooks). Le développement applicatif et la conception fonctionnelle relèvent d'autres dossiers du corpus documentaire.

FileFlow n'appelle aucun service externe pour son fonctionnement nominal.

2. Architecture de déploiement

FileFlow est distribuée sous la forme d'une **image conteneur unique**, construite en plusieurs étapes (*multi-stage build*) :

- **Étape de compilation du frontend** : génération des ressources statiques de l'interface (React).

- **Étape de compilation du backend** : production d'un **binaire Rust statique** ciblant `x86_64-unknown-linux-musl`.
- **Image finale** : socle `gcr.io/distroless/static-debian12:nonroot`, exécution sous l'utilisateur non privilégié **UID 65532**. L'image résultante pèse environ **11,3 Mo** et ne contient ni interpréteur de commandes, ni gestionnaire de paquets, ni bibliothèque système superflue.

Le service écoute par défaut sur le port **8080** et sert à la fois l'API (`/api/*`) et l'interface web (fichiers statiques servis en repli SPA).

2.1 Topologie recommandée

```
Internet ──► Reverse-proxy TLS ──► Conteneur FileFlow (port 8080)
                                        │
                                        ├── volume ./data   → /data    (fichiers utilisateurs + état applicatif)
                                        └── volume ./config → /config  (base, secret, journal d'audit)
```

La terminaison TLS est **obligatoirement** assurée par un reverse-proxy en amont (Nginx, Traefik, HAProxy ou équivalent). FileFlow n'assure pas la terminaison TLS lui-même.

2.2 Orchestration de référence (`docker-compose.yml`)

Le fichier `docker-compose.yml` fourni définit :

- un service `fileflow` basé sur l'image applicative ;
- la publication du port `8080` ;
- deux volumes persistants : `./data:/data` et `./config:/config` ;
- une politique de redémarrage `restart: unless-stopped` ;
- une sonde de santé (*healthcheck*) : commande `["/app/fileflow","healthcheck"]`, intervalle **30 s**.

---

3. Prérequis

PRÉREQUIS	DÉTAIL
Moteur de conteneurs	Docker Engine (ou runtime OCI compatible) et Docker Compose
Reverse-proxy TLS	Nginx / Traefik / HAProxy assurant HTTPS et la transmission des en-têtes
DNS	Enregistrement A/AAAA pointant vers le reverse-proxy pour le nom de service retenu
Stockage	Volume persistant dimensionné selon la volumétrie de fichiers attendue (voir §7)
Horloge	Synchronisation NTP (cohérence de l'horodatage des jetons, des liens de partage et de l'audit)

---

4. Procédure d'installation et première configuration

> **Préambule** : l'exploitation en production impose HTTPS. Les recommandations ci-dessous supposent un reverse-proxy TLS en amont.

1. **Préparer l'arborescence hôte** et les volumes :

```
mkdir -p ./data ./config
```

2. **Renseigner les variables d'environnement** (fichier `.env` ou section `environment` du compose) — voir §5. Au minimum : `ADMIN_USERNAME` et `ADMIN_PASSWORD`.
3. **Définir le secret d'administration** : `ADMIN_PASSWORD` est **requis au premier démarrage** ; il sert à créer le compte administrateur initial. Il doit être robuste (aléatoire, ≥ 16 caractères) et retiré des variables après le premier démarrage réussi.
4. **Positionner les paramètres de sécurité** :
   - `COOKIE_SECURE=true` en production (obligatoire derrière TLS). Ne passer à `false` que pour un accès local en HTTP clair, jamais en production.
   - `TRUST_PROXY_HEADERS=true` **uniquement** si le reverse-proxy en amont réécrit de manière fiable `X-Forwarded-For` / `X-Real-IP` ; sinon laisser `false` pour empêcher l'usurpation d'adresse IP dans l'audit et la limitation de débit.
5. **Gérer le secret JWT** : si `JWT_SECRET` n'est pas fourni, FileFlow **génère un secret aléatoire au premier démarrage** et le persiste dans `CONFIG_DIR/.fileflow_secret` (permissions `0600`). Pour un déploiement multi-instances ou piloté par un coffre de secrets, fournir explicitement `JWT_SECRET`.
6. **Démarrer le service** :

   ```
   docker compose up -d
   ```

7. **Vérifier la disponibilité** :

   ```
   curl -fsS http://127.0.0.1:8080/api/health
   ```

   Réponse attendue : statut `ok` et version.
8. **Retirer `ADMIN_PASSWORD`** des variables persistantes après le premier démarrage réussi, puis se connecter et modifier le mot de passe administrateur via l'interface.

---

5. Configuration de référence

VARIABLE	RÔLE	VALEUR PAR DÉFAUT	RECOMMANDATION PRODUCTION
`ROOT_DIR`	Racine des fichiers utilisateurs et de l'état applicatif (`.fileflow/`)	`/data`	`/data` (volume persistant dédié)
`CONFIG_DIR`	Emplacement de la base, du secret et du journal d'audit	Valeur de `ROOT_DIR` (le compose de référence fixe `/config`)	`/config` (volume persistant sauvegardé, distinct de `ROOT_DIR`)
`ADMIN_USERNAME`	Identifiant du compte administrateur initial	`admin`	Valeur non triviale
`ADMIN_PASSWORD`	Mot de passe du compte administrateur initial (requis au 1er démarrage)	—	Aléatoire ≥ 16 caractères, retiré après init

VARIABLE	RÔLE	VALEUR PAR DÉFAUT	RECOMMANDATION PRODUCTION
`JWT_SECRET`	Clé de signature des jetons de session	Généré et persisté si absent	Fourni via coffre de secrets si multi-instances
`MAX_UPLOAD_MB`	Taille maximale d'un téléversement (Mo)	`200`	Selon besoin métier (`50000` en compose de référence)
`PUBLIC_UPLOAD_MAX_MB`	Taille cumulée maximale d'un dépôt anonyme via une demande de téléversement (`/r/<token>`), en Mo ; toujours plafonnée à `MAX_UPLOAD_MB`	`100`	Valeur basse, ajustée au besoin réel des déposants externes
`PORT`	Port d'écoute HTTP	`8080`	`8080` (exposé via le reverse-proxy)
`COOKIE_SECURE`	Attribut `Secure` des cookies de session	`true`	`true` (obligatoire derrière TLS)
`TRUST_PROXY_HEADERS`	Prise en compte des en-têtes IP du proxy (`X-Forwarded-For` , `X-Real-IP`) pour l'audit et les verrouillages par IP	`false`	`true` seulement si proxy de confiance réécrivant ces en-têtes
`RUST_LOG`	Filtre de verbosité des journaux techniques	`fileflow=info,tower_http=warn`	Valeur par défaut ; `fileflow=debug` ponctuellement pour un diagnostic

Les variables booléennes acceptent `1` , `true` , `yes` ou `on` (insensible à la casse) ; toute autre valeur vaut `false` . Les tailles sont exprimées en mégaoctets entiers ; une valeur non numérique est ignorée au profit de la valeur par défaut.

---

6. Supervision

6.1 Sonde de santé

- Endpoint `/api/health` — **non authentifié**, renvoie un statut `ok` et la version applicative.
- La sonde conteneur exécute `["/app/fileflow","healthcheck"]` toutes les 30 s (l'image distroless ne dispose ni de shell ni de `curl` ; la sonde est embarquée dans le binaire).
- Intégrer `/api/health` à la supervision externe (reverse-proxy, orchestrateur, sonde applicative).

6.2 Journaux

- Le backend émet des journaux structurés via la bibliothèque `tracing` (sortie standard du conteneur). Les collecter via le pilote de journalisation du moteur de conteneurs ou un agent de collecte.

- Le **journal d'audit métier** est persistant : `CONFIG_DIR/.fileflow_audit.json` (format JSON par ligne), avec rotation vers `.fileflow_audit.json.1`.

6.3 Indicateurs

La définition des indicateurs de niveau de service (SLI), des objectifs (SLO) et des règles d'alerte fait l'objet du **document 11 — Observabilité (SLI/SLO)**.

6.4 Tâches de maintenance automatiques

FileFlow exécute en interne, **toutes les 30 minutes**, un cycle de maintenance : purge des sessions de téléversement inactives (et de leurs fichiers temporaires `.part`), suppression des liens de partage expirés, et nettoyage des compteurs de limitation de débit. La **corbeille est purgée automatiquement au bout de 30 jours**. Aucune tâche planifiée externe n'est requise pour ces opérations.

---

7. Sauvegarde et restauration

7.1 Éléments à sauvegarder

ÉLÉMENT	EMPLACEMENT	CRITICITÉ
Base de données applicative	`CONFIG_DIR/.fileflow.db` **ainsi que** `.fileflow.db-wal` et `.fileflow.db-shm`	Critique
Secret de signature JWT	`CONFIG_DIR/.fileflow_secret`	Critique
Journal d'audit	`CONFIG_DIR/.fileflow_audit.json` (+ `.1`)	Importante
Fichiers utilisateurs	`ROOT_DIR` (hors sous-dossier `.fileflow/`)	Critique
État applicatif fichiers	`ROOT_DIR/.fileflow/` (versions, corbeille, `metadata.json`)	Importante

7.2 Cohérence WAL

La base fonctionne en mode **WAL (Write-Ahead Logging)**. Une sauvegarde par simple copie de fichiers doit impérativement inclure les fichiers annexes `-wal` et `-shm`, faute de quoi les dernières écritures seraient perdues. Pour une sauvegarde à chaud garantie cohérente, privilégier la commande `sqlite3 <base> ".backup <cible>"` qui produit un instantané transactionnellement cohérent sans interrompre le service.

7.3 Procédure de sauvegarde à chaud

1. Sauvegarder la base de manière cohérente :

   ```
   sqlite3 /config/.fileflow.db ".backup /backup/fileflow-$(date +%F).db"
   ```

2. Copier le secret et le journal d'audit :

   ```
   cp /config/.fileflow_secret /config/.fileflow_audit.json* /backup/
   ```

3. Synchroniser l'arborescence des fichiers utilisateurs et l'état applicatif :

   ```
   rsync -a --delete /data/ /backup/data/
   ```

4. Chiffrer et externaliser l'archive de sauvegarde (le secret JWT et les fichiers utilisateurs sont sensibles).

5. Vérifier périodiquement la restaurabilité (test de restauration régulier).

7.4 Procédure de restauration

1. Arrêter le service : `docker compose down`.
2. Restaurer les volumes `config` et `data` depuis la sauvegarde (base, `-wal` / `-shm` éventuels, secret, audit, fichiers).
3. Vérifier les permissions : propriété **UID 65532**, secret en `0600`.
4. Redémarrer : `docker compose up -d`.
5. Contrôler `/api/health`, puis vérifier la connexion et l'accès à quelques fichiers.

> **Point d'attention** : restaurer une base **sans** le secret JWT d'origine invalide toutes les sessions existantes (voir Runbook R2). C'est un comportement attendu et non bloquant, mais les utilisateurs devront se reconnecter.

---

8. Plan de reprise et de continuité (PRA/PCA)

PARAMÈTRE	VALEUR INDICATIVE	COMMENTAIRE
RPO (perte de données maximale admissible)	≤ 24 h	Aligné sur une sauvegarde quotidienne ; à réduire par sauvegardes plus fréquentes selon la criticité
RTO (délai de reprise visé)	≤ 2 h	Restauration de volumes + redémarrage conteneur sur une plateforme saine

8.1 Stratégie de continuité

- **Sauvegardes** externalisées et chiffrées, testées régulièrement.
- **Bascule** : l'application étant *stateless* hormis ses volumes, la reprise consiste à instancier le conteneur sur un hôte sain et à y raccrocher les volumes restaurés.
- **Multi-instances** : un déploiement en haute disponibilité impose un `JWT_SECRET` partagé explicite et un stockage partagé cohérent ; la base SQLite mono-fichier n'étant pas conçue pour des écritures concurrentes multi-nœuds, ce scénario relève d'une évolution d'architecture (voir document 12 — registre de dette).

---

9. Gestion des montées de version

1. **Construire** la nouvelle image applicative à partir des sources versionnées.
2. **Sauvegarder** l'état (base, secret, fichiers) selon §7 **avant** toute montée de version.
3. **Migration des données** : au démarrage, FileFlow importe automatiquement et une seule fois les éventuels fichiers de configuration hérités au format JSON (`.fileflow_*.json`) vers la base SQLite si les tables correspondantes sont vides ; aucune action manuelle n'est requise.
4. **Déployer** la nouvelle image (`docker compose up -d`).
5. **Vérifier** `/api/health`, la connexion et un échantillon de fonctionnalités.
6. **Rollback** : en cas d'anomalie, redéployer l'image précédente et, si nécessaire, restaurer la sauvegarde prise à l'étape 2. Conserver au moins la version précédente de l'image et la dernière sauvegarde valide.

---

10. Sécurité d'exploitation

- **Moindre privilège** : le conteneur s'exécute sous un utilisateur non privilégié (UID 65532) sur un socle distroless minimal (surface d'attaque réduite, aucune primitive shell).
- **Permissions des secrets** : `.fileflow.db` et `.fileflow_secret` sont restreints en `0600` ; contrôler ces permissions après toute restauration ou intervention manuelle.
- **Rotation du secret JWT** : la rotation invalide l'ensemble des sessions actives (voir Runbook R2). La planifier lors d'une fenêtre de maintenance.
- **TLS et cookies** : `COOKIE_SECURE=true` et TLS de bout en bout sont impératifs en production.
- **En-têtes de proxy** : ne faire confiance aux en-têtes IP (`TRUST_PROXY_HEADERS=true`) que derrière un proxy maîtrisé, sous peine de neutraliser la limitation de débit et de fausser l'audit.
- **Cloisonnement** : les fichiers internes (`.fileflow*`) sont inaccessibles via l'API ; ne jamais exposer directement les volumes.
- **Verrouillages anti-force-brute** : la connexion, la vérification 2FA et les mots de passe des liens publics sont protégés par des verrous temporaires de 15 minutes (voir R8 et R9). Ces verrous sont tenus **en mémoire** : ils sont purgés périodiquement et **levés par un redémarrage** du service. Un redémarrage n'est donc pas une action anodine pendant une attaque en cours.
- **Dernier administrateur** : l'application refuse de rétrograder, désactiver ou supprimer le dernier administrateur **actif** (voir R10). Maintenir au moins deux comptes administrateurs actifs nominatifs.
- **Chaîne d'approvisionnement** : les dépendances (Rust, frontend, actions CI, images Docker) sont surveillées par Dependabot et par les contrôles du workflow `security` (voir R11 et R12). Les exceptions d'avis de sécurité justifiées sont tracées dans `backend/.cargo/audit.toml` et `backend/deny.toml` .

---

11. Runbooks

> Chaque runbook est actionnable et suppose un accès à l'hôte d'exécution et aux volumes. Consigner chaque intervention dans l'outil de suivi d'incidents.

R1 — Service indisponible / healthcheck au rouge

1. Vérifier l'état du conteneur : `docker compose ps` et `docker inspect --format '{{.State.Health.Status}}' fileflow` .
2. Consulter les journaux récents : `docker compose logs --tail=200 fileflow` .
3. Tester la sonde depuis l'hôte : `curl -fsS http://127.0.0.1:8080/api/health` .
4. Si le conteneur est arrêté ou en boucle de redémarrage, identifier la cause dans les journaux :
   - Erreur d'ouverture de la base → voir **R3** ;
   - Volume `config` / `data` non monté ou permissions incorrectes → corriger le montage et la propriété (UID 65532) ;
   - `ADMIN_PASSWORD` manquant au tout premier démarrage → le renseigner.
5. Relancer : `docker compose up -d` .
6. Vérifier la santé et l'accès applicatif ; escalader si l'indisponibilité persiste au-delà du RTO.

R2 — Perte ou rotation du secret JWT

> **Impact** : tout changement du secret **invalide immédiatement toutes les sessions actives**. Les utilisateurs devront se reconnecter. Aucune donnée n'est perdue.

1. Communiquer la fenêtre de maintenance aux utilisateurs.
2. Arrêter le service : `docker compose down`.
3. Selon le cas :
   - **Rotation planifiée** : supprimer `CONFIG_DIR/.fileflow_secret` (un nouveau secret sera généré au démarrage) **ou** fournir un nouveau `JWT_SECRET` explicite.
   - **Perte du secret** : le service génère automatiquement un nouveau secret au démarrage suivant ; aucune récupération de l'ancien n'est possible ni nécessaire.
4. Redémarrer : `docker compose up -d`.
5. Vérifier `/api/health`, se connecter et confirmer que l'émission de nouvelles sessions fonctionne.
6. Contrôler que le nouveau secret est bien en `0600`.

R3 — Restauration après corruption de la base

1. Arrêter le service : `docker compose down`.
2. Diagnostiquer l'intégrité : `sqlite3 /config/.fileflow.db "PRAGMA integrity_check;"`.
3. Si l'intégrité est compromise :
   1. Mettre de côté les fichiers corrompus (`.fileflow.db`, `-wal`, `-shm`).
   2. Restaurer la dernière sauvegarde cohérente de la base (§7.4).
   3. Vérifier la présence et les permissions du secret et des fichiers utilisateurs.
4. Redémarrer : `docker compose up -d`.
5. Vérifier `/api/health`, la connexion, la liste des utilisateurs et l'accès à des fichiers témoins.
6. Analyser l'origine de la corruption (arrêt brutal, saturation disque — voir **R4**, panne stockage) et corriger la cause racine.

R4 — Espace disque saturé (uploads / versions / corbeille)

1. Constater l'occupation : `df -h` sur l'hôte, puis détailler le volume `data` :

   ```
   du -sh /data/.fileflow/versions /data/.fileflow/trash /data/.fileflow/uploads
   ```

2. Actions immédiates, par ordre de sûreté :
   1. Supprimer les fichiers temporaires de téléversement orphelins : purge des `.part` dans `/data/.fileflow/uploads` (ces fichiers sont également nettoyés automatiquement au démarrage et par le cycle de maintenance).
   2. Vider la corbeille via l'interface d'administration (purge automatique à 30 jours ; une purge manuelle anticipée est possible).
   3. Évaluer la volumétrie des **versions** de fichiers dans `/data/.fileflow/versions`.
3. À moyen terme : augmenter la capacité du volume, ajuster `MAX_UPLOAD_MB`, définir une politique de rétention des versions.
4. Vérifier que le service repasse au vert (`/api/health`) une fois l'espace libéré.

R5 — Compromission suspectée

1. **Contenir** : si nécessaire, isoler le service (retrait du reverse-proxy) le temps de l'analyse.
2. **Révoquer les sessions** :
   - Révocation ciblée d'un utilisateur : action « révoquer les sessions » depuis l'administration (incrémente la version de jeton de l'utilisateur).

    - Révocation globale : rotation du secret JWT (voir **R2**), qui invalide toutes les sessions.
3. **Réinitialiser les mots de passe** des comptes concernés (voir **R6** pour l'administrateur) ; toute réinitialisation de mot de passe invalide également les sessions du compte visé.
4. **Analyser le journal d'audit** `CONFIG_DIR/.fileflow_audit.json` (+ `.1`) : rechercher les événements d'authentification anormaux, les créations/suppressions de comptes, les accès et partages inhabituels, les adresses IP suspectes.
5. **Vérifier l'intégrité** des comptes (rôles, appartenances aux groupes) et des liens de partage actifs ; supprimer les partages illégitimes.
6. **Notifier** la fonction sécurité et, en cas de violation de données personnelles, déclencher **R7**.
7. **Restaurer** depuis une sauvegarde saine si une altération des données est avérée (§7.4).

R6 — Réinitialisation du mot de passe administrateur

1. **Voie nominale** : un autre compte administrateur réinitialise le mot de passe via l'administration. La réinitialisation invalide les sessions du compte visé.
2. **Aucun autre administrateur disponible** :
    1. Sauvegarder l'état (§7).
    2. Vérifier qu'aucun compte administrateur exploitable ne subsiste (l'application interdit de supprimer, rétrograder ou désactiver le dernier administrateur actif — voir R10 —, mais la perte du mot de passe ou du second facteur reste possible).
    3. Procéder à une réinitialisation contrôlée du compte administrateur initial en cohérence avec la procédure interne d'accès de secours, puis forcer un changement de mot de passe à la première connexion.
3. Confirmer la reconnexion administrateur et consigner l'intervention.

R7 — Notification de violation de données

1. Qualifier l'incident (nature, périmètre, données personnelles concernées) à partir de l'analyse **R5** et du journal d'audit.
2. Appliquer la procédure de notification définie au **document 07 — Dossier de conformité RGPD** (registre des traitements, obligations de notification à l'autorité de contrôle et, le cas échéant, aux personnes concernées).
3. Respecter les délais réglementaires de notification.
4. Documenter la chronologie, les mesures de confinement et de remédiation, et le retour d'expérience.

R8 — Utilisateur bloqué à la connexion (réponse 429)

Modèle de verrouillage (`backend/src/ratelimit.rs`) : chaque échec de mot de passe est compté sur trois clés, chacune verrouillée **15 minutes** une fois son seuil atteint :

NIVEAU	CLÉ	SEUIL	REMISE À ZÉRO
Adresse IP	IP du client	5 échecs	Connexion réussie depuis cette IP
Couple utilisateur + IP	(identifiant, IP)	5 échecs	Connexion réussie de ce couple
Compte, toutes IP	identifiant	50 échecs	**Aucune** : expire seul 15 min après le dernier échec

Un tiers ne peut donc plus bloquer un compte avec cinq mauvais mots de passe : il lui faut environ 50 échecs, répartis sur au moins 10 adresses IP. La vérification 2FA dispose de son propre verrou par utilisateur (5 échecs → 15 min).

1. **Qualifier** l'événement dans le journal d'audit (administration → journal, filtre « action », ou directement dans `CONFIG_DIR/.fileflow_audit.json`) :
   - `login_failed` (détail `invalid credentials` ou `account disabled`) : échecs ayant alimenté les compteurs, avec IP ;
   - `login_blocked_ip` : tentative rejetée parce que l'**IP** est verrouillée ;
   - `login_blocked_user` : tentative rejetée parce que le **couple utilisateur + IP** ou le **compte** est verrouillé ;
   - `2fa_verify_failed`, `2fa_recovery_failed`, `2fa_blocked` : équivalents pour l'étape 2FA.

   ```
   grep -E '"action":"login_(failed|blocked_ip|blocked_user)"'
   /config/.fileflow_audit.json | tail -50
   ```

2. **Distinguer** l'erreur légitime de l'attaque : quelques échecs depuis l'IP habituelle de l'utilisateur → erreur de saisie ; nombreux échecs depuis des IP variées ou sur de nombreux identifiants → attaque (passer à **R5** si un compte a pu être compromis).
3. **Vérifier la résolution d'IP** : si toutes les entrées portent la même IP (celle du reverse-proxy), `TRUST_PROXY_HEADERS` est probablement à `false` derrière un proxy, et tous les utilisateurs partagent alors le même compteur IP. Corriger la configuration (§5) plutôt que de redémarrer.
4. **Débloquer** : le plus sûr est d'attendre l'expiration (15 minutes). Il n'existe pas d'action de déblocage ciblée ; un **redémarrage** du service lève **tous** les verrous (ils sont en mémoire), y compris ceux d'un attaquant : à réserver au cas d'un blocage légitime avéré, hors attaque en cours.
5. Si le compte est désactivé (`account disabled`), le réactiver depuis l'administration après validation.

R9 — Lien public verrouillé (réponse 429)

1. **Constat** : un destinataire reçoit `429` en saisissant le mot de passe d'un lien de partage (`/api/s/<token>`) ou d'une demande de téléversement (`/api/r/<token>`).
2. **Cause** : 5 mots de passe erronés en 15 minutes, comptés sur deux clés : le **lien** (toutes IP confondues) et l'**adresse IP** du client (tous liens confondus). Un succès n'efface que le compteur du lien. Ces échecs ne sont pas journalisés dans l'audit.
3. **Action** : attendre 15 minutes, puis communiquer le mot de passe par un canal fiable. Si le lien est volontairement saturé par un tiers (verrouillage de la clé « lien » pour tous — point ouvert N1, voir document 12), le propriétaire peut supprimer le lien et en créer un nouveau (nouveau jeton, compteur vierge). Un redémarrage lève également ces verrous.
4. **Rappels utiles au support** :
   - un lien répond `404` (sans distinction) s'il est expiré, a atteint son plafond de téléchargements, si son créateur est désactivé ou a perdu l'accès en lecture, ou si le fichier a été supprimé ;
   - un lien avec plafond de téléchargements ignore les reprises (`Range`) : chaque téléchargement est complet et compte pour un ;
   - un dépôt public doit envoyer le mot de passe **avant** les fichiers (formulaire fourni par FileFlow conforme) ; sa taille est plafonnée par `PUBLIC_UPLOAD_MAX_MB`.

R10 — Refus de modification du dernier administrateur (réponse 400)

1. **Constat** : la rétrogradation, la désactivation ou la suppression d'un administrateur échoue avec le message « Cannot demote or disable the last active admin » ou « Cannot delete the last admin account ».
2. **Cause** : c'est le dernier administrateur **actif** (un administrateur désactivé ne compte pas). Le contrôle est fait de façon atomique avec la modification.
3. **Action** : créer ou réactiver d'abord un autre administrateur nominatif, vérifier qu'il peut se connecter, puis rejouer l'opération. Ne jamais contourner la protection par une modification directe de la base.

R11 — Traitement des mises à jour Dependabot

Dependabot (`.github/dependabot.yml`) propose chaque **lundi** les mises à jour des dépendances Rust (`cargo`), frontend (`npm` /pnpm), des actions CI (épinglées par empreinte de commit) et des images Docker. Les mises à jour mineures et correctives sont **groupées** ; seules les versions publiées depuis au moins **7 jours** sont proposées (délai de carence). Les alertes et correctifs de sécurité sont activés sur le dépôt et ignorent ce délai.

1. **Trier** les PR : correctifs de sécurité en priorité, puis groupes mineurs/correctifs, puis montées majeures.
2. **Fusionner** une PR uniquement si les workflows `ci` et `security` sont au vert.
3. **Montées majeures** (changements d'API possibles) : ne pas fusionner en l'état ; les instruire comme un chantier (branche dédiée, adaptation du code, tests) et les suivre dans le registre de dette (document 12).
4. **Image Docker** : après fusion d'une mise à jour d'image, reconstruire l'image applicative et la redéployer selon §9.
5. **Avis non corrigeable** : si un avis concerne un code non atteignable et qu'aucune version corrigée n'existe, documenter l'exception (identifiant, justification) dans `backend/.cargo/audit.toml` **et** `backend/deny.toml` , puis la réexaminer à chaque nouvelle version de la dépendance.

R12 — Échec d'un contrôle de sécurité en CI

Le workflow `security` s'exécute à chaque push et pull request, et chaque lundi à 06:00 UTC. Le workflow `ci` exécute en outre ESLint (`pnpm lint`) et `cargo clippy -D warnings` , tous deux bloquants.

JOB	CONTRÔLE	CONDUITE À TENIR EN CAS D'ÉCHEC
`sast`	Analyse statique semgrep	Corriger le code ; une suppression `nosemgrep` n'est admise qu'avec justification en commentaire
`cargo-audit`	Avis RustSec sur `Cargo.lock`	Mettre à jour la dépendance ; à défaut, exception justifiée (R11 étape 5)
`cargo-deny`	Avis, licences (liste blanche), sources (crates.io uniquement), crates bannies	Remplacer la dépendance non conforme ou documenter l'exception dans `deny.toml`
`pnpm-audit`	Avis sur les dépendances frontend de production	Mettre à jour le paquet concerné
`trivy-fs`	Vulnérabilités et mauvaises configurations du dépôt (CRITICAL, HIGH, MEDIUM)	Corriger ou mettre à jour

JOB	CONTRÔLE	CONDUITE À TENIR EN CAS D'ÉCHEC
`trivy-image`	Vulnérabilités de l'image construite (CRITICAL, HIGH)	Reconstruire sur une image de base à jour
`gitleaks`	Secrets commités	Révoquer et renouveler **immédiatement** le secret exposé, puis le retirer de l'historique
`fuzz`	Fuzzing des 4 cibles (hebdomadaire uniquement)	Reproduire avec l'entrée fournie, corriger, ajouter un test de non-régression

Un échec hebdomadaire sans modification du code signale en général un nouvel avis publié sur une dépendance existante : le traiter selon R11.

---

12. Références

- **ITIL 4**, Axelos — pratiques de gestion des services (gestion des incidents, des changements, de la continuité).
- **Google, « Site Reliability Engineering »**, O'Reilly, 2016 — chapitres relatifs aux *runbooks* / *playbooks* et à la gestion opérationnelle.
- **Document 07** — Dossier de conformité RGPD (corpus documentaire FileFlow).
- **Document 11** — Observabilité (SLI/SLO) (corpus documentaire FileFlow).
- **Document 12** — Registre des dettes techniques (corpus documentaire FileFlow).
- **Document 15** — Plan directeur des scans et rapports de sécurité (corpus documentaire FileFlow).